Adaptive Poker Bots: Crisp Rules vs Fuzzy Logic vs Learning

COMP 3106 A Fall 2026: Project Proposal – November 5, 2026

Team: Yufeng Liu, 101258905 · Thomas Nolasque, 101266644 · Teja Chilukuri, 101261938

1. Background and objectives

No-limit Texas hold'em is a game of imperfect information. Players cannot see opponents' cards, outcomes are stochastic, and the best action depends on how the specific opponents at the table behave. A fixed strategy that beats one opponent style is often exploited by another, so **adapting to opponents** is central to strong play.

We will build three poker agents, each grounded in a different AI paradigm, and pit them against each other at a multi-player no-limit table, with the remaining seats filled by simple scripted bots. Our objectives are to:

1. Implement a **crisp rule-based expert system**, a **fuzzy rule-based system**, and a **machine-learning agent** that play complete hands of multi-player no-limit hold'em.
2. All three agents will derive their own opponent-modelling information, and measure **how well and how quickly each adapts** to different opponent styles.
3. Compare the agents on overall strength (chips won) and on adaptation speed under controlled, fair conditions.
4. Deliver a local demo in which a human joins the table in a browser and sees each bot's reasoning for its actions.

Some definitions regarding what consists of a game in our context:

- **Table** – Consists of hundreds to thousands of **rotations**
- **Rotations** – 8 rounds dealt from the same random seed where each bot plays every seat once
- **Round (Hand)** – From dealing the hole cards to when the pot is awarded

2. Proposed AI methods

Every bot receives the **same inputs**:

- **Hole Cards** – In the form of a tuple. Ex: (7c,Qh) = 7 of clubs, Q of hearts
- **Pot value** – In the form of Big Blinds (bb)
- **Player ID** – The model's current player ID. Randomly assigned value between 1000 – 9999. Player ID will be assigned per game, meaning if the game changes, so will player ID.
- **Position** – An array of player IDs of length 8 for the current table. playerIDs[i] where i represents the position of the player. 0 = small blind, 1 = big blind, …, 7 = button
- **Action history** – Every action played in the current hand of max length 8. 2D array, ex: [[playerID, action, amount], …, [playerID, action, amount]]
- **Chip count of all players** – An array of length 8, in Big Blind notation of each player at index i. A value of None represents a player who has busted. Ex: [10,20,7,None,30,50,80,0]
- **Community cards** – An array of length 0 (pre-flop),3 (flop),4 (turn) or 5 (river). Ex: [Ah, 9d, 2c]
- **Actions** – Array of a subset of (action, cost in bb) tuples to choose from. Fold is only offered when there is a bet to call. Actions include [("call", 10), ("raise ½ pot", 20), ("raise 1 pot", 40), ("all-in", 50), ("fold",0), ("check", 0)]
- **Table ID** – Unique identifier for tables
- **Round ID** – Unique identifier for rounds within tables

Bot A: Crisp rule-based expert system - A knowledge base of IF-THEN rules maps the inputs to actions. Example: *IF equity > pot odds AND opponent is passive AND position is late THEN raise ½ pot*. Rules fire in a fixed priority order. Adaptation comes from rules whose thresholds shift with the opponent statistics. An explanation facility reports the chain of rules that fired.

Bot B: Fuzzy rule-based system - The same inputs are fuzzified with membership functions (e.g. equity ∈ {weak, medium, strong}; opponent ∈ {tight, loose} × {passive, aggressive}). Mamdani inference over a fuzzy rule base produces an aggression score. Centroid defuzzification of that score selects the action and bet size. Adaptation comes from the opponent-style fuzzy sets, which let behaviour shift gradually instead of at hard thresholds. Explanations list the rules with the highest firing strengths.

Bot C: Machine-learning agent - A supervised classifier (gradient-boosted trees or a small neural network) trained on the shared feature vector. Training data comes from logged hands against a rotating pool of Bot A, Bot B, and the scripted opponents (Section 4), so it learns to respond to opponent statistics rather than to one fixed opponent. **Labels:** each logged decision is replayed from the same seeded state with every possible action, and labelled with the action that won the most chips. No reinforcement learning is used, which avoids reward design and reward hacking. Explanations use feature attributions such as SHAP.

3. Environment

The game is **8 seat no-limit Texas hold'em**, with 1/2 blinds and stacks reset to 100 big blinds each hand. Bots A, B, and C sit at every table, and the remaining seats are filled by scripted bots (Section 4) drawn at random for each session. We will use RLCard's no-limit hold'em environment for game logic and the treys library for fast hand evaluation. We will build these shared components ourselves:

- A common `Bot` interface with `act()`, `explain()`, and `observe_result()` (called after every round, including folded rounds. Updates opponent model)
- The bots' model of each opponent (opponent model)
- A tournament harness

- A local web page where a human can take a seat and play or observe a table

An external dataset is not needed. All training and evaluation data is generated through self-play, and hands are logged for analysis.

All bots get identical observations and the same opponent-model module, a 200 ms limit per decision, and an equal tuning budget against the same practice opponents. The bots differ only in how they reason, not in what they know. Every round also rotates each bot through every seat with identical cards (Section 4), so no bot benefits from better cards or position.

4. Validation and analysis

Opponents. Simple scripted bots fill the empty seats and act as test opponents:

1. **Tight-aggressive (TAG):** plays only the top ~15% of starting hands, and always raises
2. **Tight-passive (TP):** plays only the top ~10% of starting hands, and always calls
3. **Loose-aggressive (LAG):** plays top ~40% of starting hands and always raises
4. **Calling machine (always-call):** plays top ~50% of starting hands and always calls
5. **Always-all-in:** goes all-in every time it can act
6. **Random:** picks a legal action uniformly at random
7. **Switcher:** Rotates between one of the five (excluding random) above models every 25 rotations to test model adaptation.

Experiments.

1. **Chip tournament (main result).** Rank the bots by total chips at the end of every table. Chips carry across all tables, so luck from cards and seating cancels out.
2. **Adaptation.** Track each bot's chip total over time against each scripted bot. Against the Switcher, count how many hands after the switch it takes each bot to start winning chips again.
3. **Opponent-model check.** Re-run experiment 2 with the opponent model turned off, to show how many chips each bot gains from adapting.

Qualitative analysis. We will compare the explanations each bot gives in identical game states, and report hands where the three paradigms disagree.

5. Novelty

Most poker-bot projects focus on one method against fixed opponents. Our project differs in four ways:

1. **Controlled comparison across three paradigms.** Crisp rules, fuzzy rules, and learning are compared with identical inputs and opponent knowledge, so differences reflect the reasoning method alone.
2. **Adaptation as the measured outcome.** We measure adaptation directly with a strategy-switching opponent and an opponent-model ablation, instead of reporting only final chip counts.
3. **Crisp vs fuzzy on a shared feature set.** This tests whether fuzzy boundaries give smoother adaptation than hard thresholds.
4. **Multi-player no-limit setting with explanations.** Each bot outputs its actions during live play.

6. Schedule of milestones

Each member owns one bot (A,B or C) and some shared components.

Week	Milestone
Nov 9–15	• GitHub repo set up and Setup issues and tasks for team members • RLCard wrapper and Bot interface • equity calculator • Simple scripted opponents built
Nov 16–22	• First playable version of Bots A, B, and C • tournament harness with seeded seat-rotation rounds and chip tracking • first labelled training data for Bot C
Nov 23–29	• Rule and membership-function tuning • Bot C classifier training and feature tuning • explain() for all bots
Nov 30–Dec 6	• Full experiments 1–3 • figures • web demo table • report draft
Dec 7–11	• Demo rehearsal and presentation • final report • README and code clean-up • submit